

Construcción de Consultas SQL a Tablas

Una guía completa para dominar el arte de extraer información valiosa de bases de datos relacionales mediante consultas estructuradas.

El rol de SQL en el análisis de datos

El Lenguaje Estructurado de Consultas (SQL, Structured Query Language) es la herramienta estándar para interactuar con bases de datos relacionales. Permite consultar, insertar, modificar y eliminar información. En el contexto del análisis de datos, la consulta de información mediante SQL es un paso crucial: es aquí donde se extraen las piezas de información necesarias para transformar grandes volúmenes de datos en insights accionables.

Traducción de requerimientos

Un analista que domina SQL puede traducir requerimientos de negocio en consultas estructuradas que recuperan información con precisión.

Extracción precisa

Permite obtener exactamente los datos necesarios sin información redundante o irrelevante.

Insights accionables

Convierte grandes volúmenes de datos en información útil para la toma de decisiones.

Preguntas de negocio que SQL puede resolver

Por ejemplo, responder preguntas como:



Análisis de ventas

¿Cuál es el promedio de ventas mensuales por región?



Comparación de productos

¿Qué productos tienen un precio superior al promedio del mercado?



Análisis hospitalario

¿Cuántos pacientes fueron atendidos por cada especialidad en un hospital?

Todas estas preguntas pueden ser resueltas con consultas SQL bien estructuradas.

Consultas simples y sintaxis básica de SELECT

La consulta más básica en SQL utiliza el comando SELECT. La sintaxis general es:

```
SELECT columnas  
FROM tabla  
[WHERE condiciones]  
[GROUP BY columnas]  
[HAVING condición]  
[ORDER BY columnas];
```

Ejemplo

Supongamos que tenemos una tabla empleados con los siguientes campos:

- id_empleado
- nombre
- departamento
- salario

Ejemplos básicos de SELECT

Una consulta simple que obtiene todos los registros es:

```
SELECT *  
FROM empleados;
```

Esto devuelve todas las columnas y filas de la tabla.

Para obtener solo nombres y salarios:

```
SELECT nombre, salario  
FROM empleados;
```

En este punto, el analista empieza a filtrar lo que realmente es relevante.

Implementación de lógica con WHERE

La cláusula WHERE permite filtrar registros de acuerdo con condiciones lógicas.

Ejemplo práctico

Extraer empleados con salario mayor a 2.000.000:

```
SELECT nombre, salario  
FROM empleados  
WHERE salario > 2000000;
```

Si el requerimiento es empleados del área de ventas con salario superior a 1.500.000:

```
SELECT nombre, departamento, salario  
FROM empleados  
WHERE departamento = 'Ventas'  
AND salario > 1500000;
```

Aquí ya se observa cómo los criterios de negocio se transforman en condiciones SQL.

Operadores condicionales en las consultas

SQL ofrece operadores para construir condiciones más complejas:

Comparación básica

- = igualdad
- <> o != diferente
- > mayor que
- < menor que

Comparación avanzada

- >= mayor o igual
- <= menor o igual
- BETWEEN rango

Patrones y conjuntos

- LIKE patrones en cadenas
- IN conjunto de valores

Ejemplos con BETWEEN y LIKE

Ejemplo con BETWEEN y LIKE

Seleccionar empleados con salario entre 1.000.000 y 2.000.000:

```
SELECT nombre, salario  
FROM empleados  
WHERE salario BETWEEN 1000000 AND 2000000;
```

Seleccionar empleados cuyo nombre empiece por "A":

```
SELECT nombre  
FROM empleados  
WHERE nombre LIKE 'A%';
```

En bases como PostgreSQL, también se pueden usar expresiones regulares con ~ para patrones más complejos.

Funciones de agregación: MIN, MAX, AVG, COUNT

Las funciones de agregación resumen información de conjuntos de registros.



MIN(columna)

Devuelve el valor mínimo de una columna



MAX(columna)

Devuelve el valor máximo de una columna



AVG(columna)

Calcula el promedio de los valores



COUNT(columna)

Cuenta el número de registros

Ejemplo aplicado de funciones de agregación

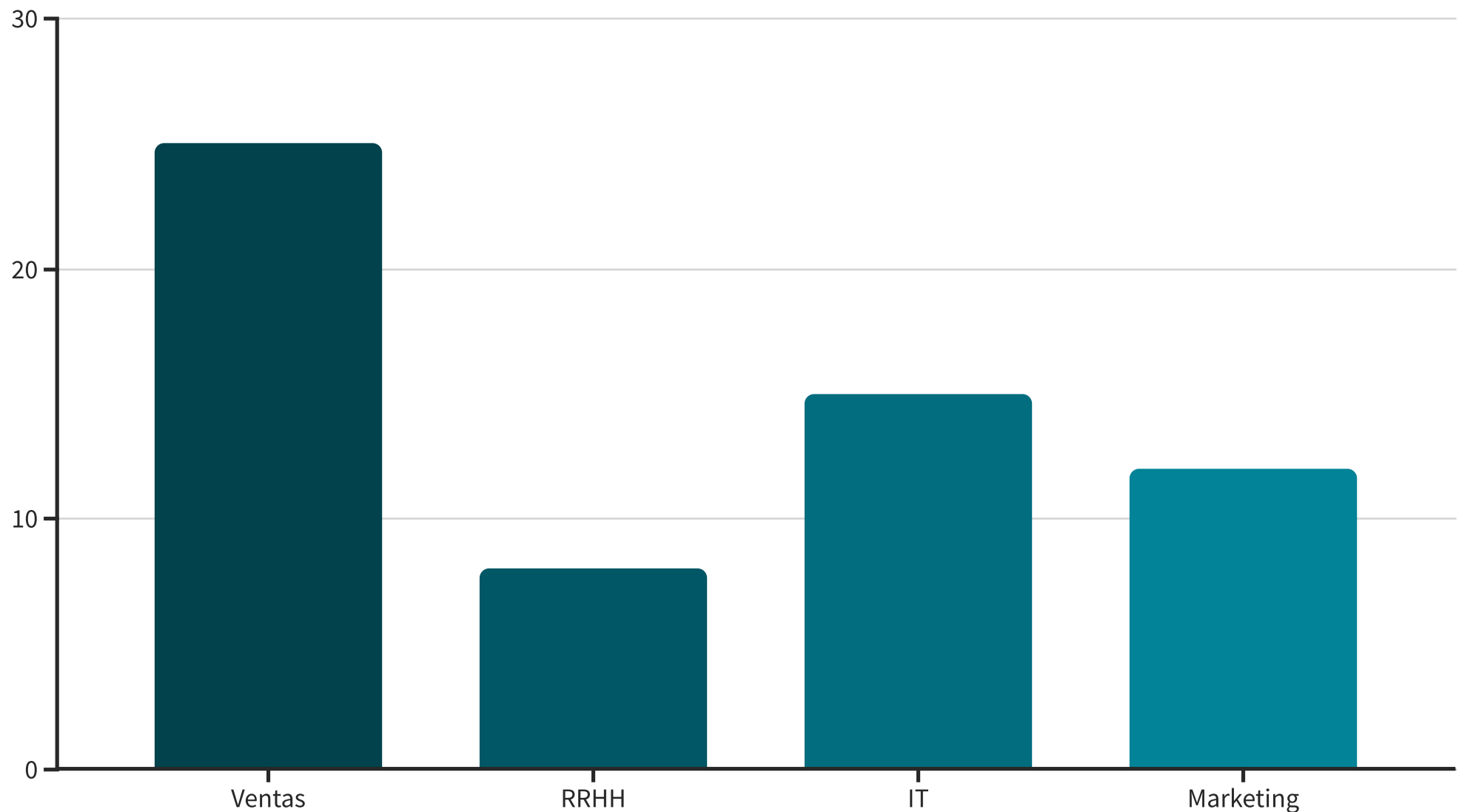
Encontrar el salario mínimo, máximo y promedio en la tabla de empleados:

```
SELECT MIN(salario) AS salario_minimo,  
       MAX(salario) AS salario_maximo,  
       AVG(salario) AS salario_promedio  
FROM empleados;
```

Contar empleados por departamento:

```
SELECT departamento, COUNT(*) AS total_empleados  
FROM empleados  
GROUP BY departamento;
```

Esto último introduce la necesidad de agrupar datos.



Operadores lógicos: AND y OR

Los operadores AND y OR permiten combinar múltiples condiciones.

AND

Todas las condiciones deben cumplirse.

| Empleados de Ventas Y con salario alto

OR

Basta con que se cumpla una condición.

| Empleados de Ventas O de RRHH

Ejemplo

Obtener empleados de Ventas con salario superior a 2.000.000 o de Recursos Humanos con salario superior a 1.800.000:

```
SELECT nombre, departamento, salario
FROM empleados
WHERE (departamento = 'Ventas' AND salario > 2000000)
OR (departamento = 'RRHH' AND salario > 1800000);
```

Esto refleja decisiones de negocio donde se cruzan reglas distintas para diferentes grupos.

Agrupamiento de registros con GROUP BY y HAVING

El comando GROUP BY agrupa filas según valores comunes en una o varias columnas. La cláusula HAVING se utiliza para filtrar esos grupos en función de condiciones sobre agregaciones.

Ejemplo práctico

Número de empleados por departamento:

```
SELECT departamento, COUNT(*) AS total  
FROM empleados  
GROUP BY departamento;
```

Filtrar solo los departamentos con más de 10 empleados:

```
SELECT departamento, COUNT(*) AS total  
FROM empleados  
GROUP BY departamento  
HAVING COUNT(*) > 10;
```

Esto permite descubrir patrones, como "qué áreas concentran más personal" o "qué sucursales superan cierto umbral de ventas".

Ejemplos de la industria



Retail

Obtener los productos más vendidos y calcular su participación sobre el total.



Banca

Contar el número de clientes con créditos activos segmentados por rango de edad.



Salud

Calcular el promedio de días de hospitalización por especialidad.



Minería

Agrupar reportes de incidentes por tipo y mostrar solo los que superan un cierto umbral.

Estos casos reflejan cómo los operadores de SQL transforman datos dispersos en indicadores clave.

Ejercicios prácticos – Parte 1

A continuación, se presentan ejercicios integrados:

01

Consulta simple

Obtener todos los clientes de una base de datos de ventas en línea:

```
SELECT * FROM clientes;
```

02

Filtro con WHERE

Clientes de Chile con más de 5 pedidos:

```
SELECT nombre, pais, pedidos  
FROM clientes  
WHERE pais = 'Chile' AND pedidos > 5;
```

03

Uso de operadores condicionales

Productos con precio entre 100 y 500 dólares:

```
SELECT nombre_producto, precio  
FROM productos  
WHERE precio BETWEEN 100 AND 500;
```

Ejercicios prácticos – Parte 2

01

Funciones de agregación

Salario promedio por departamento:

```
SELECT departamento, AVG(salario) AS promedio
FROM empleados
GROUP BY departamento;
```

02

Uso de HAVING

Departamentos con promedio salarial mayor a 2.500.000:

```
SELECT departamento, AVG(salario) AS promedio
FROM empleados
GROUP BY departamento
HAVING AVG(salario) > 2500000;
```

03

Combinación AND / OR

Pacientes hospitalizados más de 10 días en cardiología o más de 15 días en traumatología:

```
SELECT nombre_paciente, especialidad, dias_hospitalizacion
FROM pacientes
WHERE (especialidad = 'Cardiología' AND dias_hospitalizacion > 10)
OR (especialidad = 'Traumatología' AND dias_hospitalizacion > 15);
```

